SBA301 – LAB 06

SECURE REST SERVICES WITH SPRING BOOT USING JSON WEB TOKEN (JWT)

HƯỚNG DẪN THỰC HÀNH CHI TIẾT STEP BY STEP

Spring Boot 3 • Spring Security • JWT • Spring Data JPA • SQL Server • Postman

Case study: Orchid Management – Registration / Login / Protected User APIs
Course baseline: JDK 21 • Maven • Spring Boot 3.x • SQL Server • JJWT 0.12.x

CORE PRINCIPLE: Mỗi bước trong tài liệu đều phải được tự kiểm chứng. “Code compile” chưa đủ: sinh viên cần quan sát HTTP status, response body, Spring Security flow, SQL Server state và negative cases.

1. TÓM TẮT NỘI DUNG VÀ YÊU CẦU

Lab 06 xây dựng RESTful API có đăng ký và đăng nhập, dùng Spring Security để xác thực người dùng và JSON Web Token (JWT) làm bearer credential cho các request sau đăng nhập. Dữ liệu User được lưu bằng Spring Data JPA trên Microsoft SQL Server. Tài liệu giữ đúng flow của Lab 06 gốc nhưng bổ sung giải thích nền tảng, checkpoint và cách tự đánh giá ở từng bước.

1.1. Kiến thức nền cần hiểu trước khi code

Khái niệm	Ý nghĩa trong Lab 06
Authentication	Xác minh “bạn là ai?” bằng email + password hoặc JWT hợp lệ.
Authorization	Quyết định “bạn được phép làm gì?”. Core Lab chủ yếu yêu cầu authenticated user; role được đưa vào phần nâng cao.
JWT	Chuỗi token gồm Header.Payload.Signature. Payload không mặc định là bí mật; signature bảo vệ integrity/authenticity.
Bearer token	Ai “mang” token hợp lệ có thể gửi nó trong Authorization header; vì vậy token phải được bảo vệ.
UserDetails	Contract Spring Security dùng để đọc username/password/authorities/account status.
PasswordEncoder	Hash password bằng adaptive one-way function; không lưu plaintext.
SecurityFilterChain	Chuỗi filter kiểm tra request trước khi tới Controller.
SecurityContext	Nơi Spring Security giữ Authentication của request hiện tại.

1.2. Luồng hệ thống cần đạt

```
SIGNUP
Client -> POST /auth/signup -> AuthenticationController
       -> AuthenticationService.signup -> PasswordEncoder
       -> UserRepository.save -> SQL Server

LOGIN
Client -> POST /auth/login -> AuthenticationManager
       -> DaoAuthenticationProvider -> UserDetailsService + PasswordEncoder
       -> JWT generated -> token returned

PROTECTED REQUEST
Client -> GET /users/me + Authorization: Bearer <JWT>
       -> SecurityFilterChain -> JwtAuthenticationFilter
       -> verify token -> load UserDetails -> SecurityContext
       -> Authorization check -> UserController -> response
```

1.3. Kết quả tối thiểu phải đạt

- POST /auth/signup tạo User mới và DB chỉ lưu password hash.
- POST /auth/login với credential đúng trả JWT và expiresIn.
- Credential sai bị từ chối.
- GET /users/me không có token bị từ chối.
- GET /users/me có JWT hợp lệ trả đúng principal.
- JWT bị sửa hoặc hết hạn bị từ chối.
- GET /users không để lộ password/hash trong response.
- Có Postman evidence, SQL evidence và ít nhất một automated security test.

2. CÔNG NGHỆ, PHẦN MỀM VÀ ĐIỀU KIỆN CHUẨN BỊ

Thành phần	Mục đích	Cách kiểm tra
JDK 21	Compile/run Spring Boot	java -version
Maven / mvnw	Build/dependency/test	mvn -v hoặc mvnw.cmd -v
IntelliJ IDEA	IDE chính	Mở project, Maven sync OK
SQL Server + SSMS	Persistence	SELECT 1
Postman	REST/JWT testing	Create environment/collection
Git	Checkpoint / rollback	git status

```
java -version
mvn -v
git --version

# SQL Server smoke test
SELECT 1 AS ok;
```

PASS: Không có tool nào lỗi trước khi tạo project. Nếu Java/Maven/SQL Server chưa đúng, sửa môi trường trước; không debug JWT khi toolchain chưa pass.

3. CẤU TRÚC THƯ MỤC MỤC TIÊU

```
src/main/java/fu/se/lab06jwt/
├─ Lab06JwtApplication.java
├─ configurations/
│  ├─ ApplicationConfiguration.java
│  ├─ JwtAuthenticationFilter.java
│  └─ SecurityConfiguration.java
├─ controllers/
│  ├─ AuthenticationController.java
│  └─ UserController.java
├─ dtos/
│  ├─ LoginUserDto.java
│  └─ RegisterUserDto.java
├─ entities/
│  └─ User.java
├─ exceptions/
│  └─ GlobalExceptionHandler.java
├─ repositories/
│  └─ UserRepository.java
├─ responses/
│  ├─ LoginResponse.java
│  └─ UserResponse.java
└─ services/
   ├─ AuthenticationService.java
   ├─ JwtService.java
   └─ UserService.java

src/main/resources/
└─ application.properties
```

KIẾN TRÚC: Controller nhận HTTP; Service xử lý use case; Repository truy cập dữ liệu; configuration/security chịu trách nhiệm authentication pipeline. Không viết repository query trong Controller.

4. TÓM TẮT TOÀN BỘ CÁC BƯỚC THỰC HIỆN

Pha	Bước	Checkpoint chính
A. Environment	0.1–0.3	SQL Server + toolchain + Postman
B. Project	1.1–1.4	Spring Boot project build sạch
C. Persistence	2.1–2.5	DB config + User entity + Repository
D. Authentication Core	3.1–3.4	UserDetailsService + BCrypt + AuthenticationProvider
E. JWT	4.1–4.3	Generate / parse / validate token
F. Request Security	5.1–5.3	JWT filter + SecurityFilterChain
G. Auth API	6.1–6.4	Signup/login response
H. Protected API	7.1–7.3	/users/me + /users
I. Verification	8.1–8.8	Postman + SQL + negative matrix

Pha	Bước	Checkpoint chính
J. Quality	9–13	Trace, debugging, automated tests, gate, advanced track

5. ACTIVITY 0 – CHUẨN BỊ SQL SERVER, TOOLCHAIN VÀ POSTMAN

Step 0.1 – Tạo database riêng cho Lab 06

MỤC TIÊU: Có database độc lập để User/JPA không lẫn dữ liệu lab khác.

KIẾN THỨC NỀN CẦN NHỚ: Database name là một phần của datasource URL. Nếu DB sai tên, Spring Boot fail ngay ở bootstrap.

THAO TÁC:

- Mở SSMS.
- Kết nối SQL Server instance.
- Mở New Query và chạy script bên dưới.

CODE / CẤU HÌNH:

```
IF DB_ID(N'OrchidJWTLab') IS NULL
    CREATE DATABASE OrchidJWTLab;
GO
USE OrchidJWTLab;
GO
SELECT DB_NAME() AS current_db;
```

CÁCH CHẠY: Execute trong SSMS.

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH: Kết quả query cuối phải là OrchidJWTLab.

PASS / FAIL: PASS: DB tồn tại và query được. FAIL: login/permission/instance error thì sửa SQL Server trước.

LỖI THƯỜNG GẶP / CÁCH XỬ LÝ: Nếu dùng named instance hoặc port khác 1433, ghi lại chính xác để dùng ở application.properties.

Step 0.2 – Kiểm tra JDK/Maven/port

MỤC TIÊU: Loại bỏ lỗi môi trường trước khi viết code.

KIẾN THỨC NỀN CẦN NHỚ: Spring Boot 3 course baseline dùng Jakarta namespace và cần JDK tương thích; Lab này khóa JDK 21.

CODE / CẤU HÌNH:

```
java -version
mvn -v
netstat -ano | findstr :8005
```

CÁCH CHẠY: Chạy trong CMD/PowerShell.

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH: Java 21; Maven chạy; port 8005 không bị process khác LISTENING hoặc biết PID để xử lý.

PASS / FAIL: PASS khi toolchain đúng. FAIL nếu IntelliJ dùng JDK khác: File > Project Structure > Project SDK.

LỖI THƯỜNG GẶP / CÁCH XỬ LÝ: Không nhầm JRE/JDK; nếu Maven chạy bằng JDK khác IntelliJ, kiểm JAVA_HOME/PATH.

Step 0.3 – Tạo Postman Environment

MỤC TIÊU: Chuẩn bị biến baseUrl và token để test có hệ thống.

KIẾN THỨC NỀN CẦN NHỚ: Environment giúp giảm copy/paste token và tránh gửi nhầm server.

THAO TÁC:

- Postman > Environments > Create.
- Tạo baseUrl = http://localhost:8005.
- Tạo token = rỗng.
- Save environment và chọn environment đó.

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH: Trong request có thể dùng {{baseUrl}} và Postman hiển thị giá trị preview.

PASS / FAIL: PASS nếu {{baseUrl}} resolve đúng. FAIL nếu chưa chọn environment.

LỖI THƯỜNG GẶP / CÁCH XỬ LÝ: Token sẽ được tự lưu sau bước login.

6. ACTIVITY 1 – TẠO SPRING BOOT PROJECT VÀ DEPENDENCIES

Step 1.1 – Tạo Spring Initializr project

MỤC TIÊU: Tạo backend Spring Boot tối thiểu có Web, JPA, Security và SQL Server.

KIẾN THỨC NỀN CẦN NHỚ: Nên chứng minh project base chạy trước khi thêm JWT để giảm phạm vi debug.

THAO TÁC:

- IntelliJ IDEA > File > New > Project > Spring Initializr.
- Language Java, Build Maven, JDK 21, Packaging Jar.
- Group: fu.se; Artifact: lab06jwt; package: fu.se.lab06jwt.
- Chọn Spring Web, Spring Data JPA, Spring Security, MS SQL Server Driver, Validation.

CÁCH CHẠY: Create project, chờ Maven indexing hoàn tất.

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH: Không còn dependency đỏ; main application class compile.

PASS / FAIL: PASS nếu IDE/Maven sync sạch. FAIL nếu dependency resolve lỗi: kiểm mạng/proxy/settings.xml.

LỖI THƯỜNG GẶP / CÁCH XỬ LÝ: Không vội thêm JWT dependency trước khi Spring project cơ bản compile.

Step 1.2 – Bổ sung JJWT dependencies

MỤC TIÊU: Thêm thư viện tạo/parse JWT theo baseline của Lab gốc.

KIẾN THỨC NỀN CẦN NHỚ: JJWT tách API khỏi runtime implementation/Jackson serializer.

CODE / CẤU HÌNH:

```
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-api</artifactId>
  <version>0.12.6</version>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-impl</artifactId>
  <version>0.12.6</version>
  <scope>runtime</scope>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-jackson</artifactId>
  <version>0.12.6</version>
  <scope>runtime</scope>
</dependency>
```

CÁCH CHẠY: Reload Maven project.

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH: mvn dependency:tree có jjwt-api/impl/jackson 0.12.6.

PASS / FAIL: PASS nếu compile dependency resolution thành công.

LỖI THƯỜNG GẶP / CÁCH XỬ LÝ: Không copy code API của JJWT 0.11.x vào project 0.12.x; method names có khác.

Step 1.3 – Kiểm tra dependency đầy đủ và build sạch

MỤC TIÊU: Khóa checkpoint trước khi viết source security.

KIẾN THỨC NỀN CẦN NHỚ: Nếu project chưa compile ở đây, lỗi sau đó không thể quy cho JWT code.

CODE / CẤU HÌNH:

```
mvn -q -DskipTests compile
mvn dependency:tree
```

CÁCH CHẠY: Chạy tại thư mục chứa pom.xml.

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH: BUILD SUCCESS; tree có spring-boot-starter-web, data-jpa, security, validation, mssql-jdbc, jjwt.

PASS / FAIL: PASS khi build sạch. FAIL: xử lý dependency conflict/missing artifact trước.

LỖI THƯỜNG GẶP / CÁCH XỬ LÝ: Spring Security starter phải có trong runtime, spring-security-test là dependency test riêng nếu dùng MockMvc security helpers.

Step 1.4 – Tạo package structure

MỤC TIÊU: Tách responsibilities rõ ràng ngay từ đầu.

KIẾN THỨC NỀN CẦN NHỚ: Main application class nên ở root package để component scanning bao phủ sub-packages.

THAO TÁC:

- Tạo configurations, controllers, dtos, entities, exceptions, repositories, responses, services.
- Không đặt main application class trong package con sâu hơn.

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH: Project tree khớp mục 3.

PASS / FAIL: PASS nếu các package nằm dưới fu.se.lab06jwt.

LỖI THƯỜNG GẶP / CÁCH XỬ LÝ: Nếu bean không được scan, kiểm vị trí main class trước khi thêm @ComponentScan thủ công.

7. ACTIVITY 2 – CẤU HÌNH DATASOURCE, JWT PROPERTIES VÀ PERSISTENCE

Step 2.1 – Cấu hình application.properties

MỤC TIÊU: Kết nối SQL Server và khai báo JWT TTL/secret từ environment.

KIẾN THỨC NỀN CẦN NHỚ: Datasource config sai làm app fail trước Security. JWT secret không nên hard-code vào Git.

THAO TÁC:

- IntelliJ Run Configuration > Environment variables.
- Thêm DB_USER, DB_PASSWORD, JWT_SECRET.
- JWT_SECRET dùng khóa mạnh phù hợp HMAC; không dùng “secret”.

CODE / CẤU HÌNH:

```
spring.application.name=lab06-jwt
server.port=8005

spring.datasource.url=jdbc:sqlserver://localhost:1433;databaseName=OrchidJWTLab;encrypt=true;trustServerCertificat
e=true
spring.datasource.username=${DB_USER}
spring.datasource.password=${DB_PASSWORD}
spring.datasource.driver-class-name=com.microsoft.sqlserver.jdbc.SQLServerDriver

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

security.jwt.secret-key=${JWT_SECRET}
security.jwt.expiration-time=3600000
```

CÁCH CHẠY: mvn spring-boot:run

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH: App start đến trạng thái Started ...Application; không có Login failed/Cannot open database.

PASS / FAIL: PASS khi DataSource connect. FAIL: đọc root cause SQLException đầu tiên.

LỖI THƯỜNG GẶP / CÁCH XỬ LÝ: ddl-auto=update chỉ dùng lab/dev; production dùng migration tool.

Step 2.2 – Tạo User entity implements UserDetails

MỤC TIÊU: Map bảng users và cung cấp thông tin Spring Security cần cho authentication.

KIẾN THỨC NỀN CẦN NHỚ: getUsername() có thể trả email vì email là unique login identifier. getAuthorities() ở Core Lab trả empty list; role sẽ học ở phần nâng cao.

CODE / CẤU HÌNH:

```
@Entity
@Table(name = "users")
public class User implements UserDetails {
```

```
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;

    @Column(nullable = false)
    private String fullName;

    @Column(nullable = false, unique = true, length = 100)
    private String email;

    @Column(nullable = false)
    private String password;

    @CreationTimestamp
    @Column(name = "created_at", updatable = false)
    private Date createdAt;

    @UpdateTimestamp
    @Column(name = "updated_at")
    private Date updatedAt;

    @Override
    public Collection<? extends GrantedAuthority> getAuthorities() {
        return List.of();
    }

    @Override public String getUsername() { return email; }
    @Override public String getPassword() { return password; }
    @Override public boolean isAccountNonExpired() { return true; }
    @Override public boolean isAccountNonLocked() { return true; }
    @Override public boolean isCredentialsNonExpired() { return true; }
    @Override public boolean isEnabled() { return true; }

    // constructors, getters, setters
}
```

CÁCH CHẠY: Restart application để Hibernate tạo/validate bảng.

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH: SSMS: SELECT TOP 1 * FROM users chạy được; INFORMATION_SCHEMA thấy các cột.

PASS / FAIL: PASS khi app start và bảng users tồn tại. FAIL nếu reserved name/schema mismatch thì đọc generated SQL/error.

LỖI THƯỜNG GẶP / CÁCH XỬ LÝ: Không trả password trong Controller response sau này.

Step 2.3 – Tạo DTO cho signup/login

MỤC TIÊU: Tách HTTP request contract khỏi JPA entity.

KIẾN THỨC NỀN CẦN NHỚ: DTO giúp validation input và tránh mass-assignment trực tiếp vào entity.

CODE / CẤU HÌNH:

```
public class RegisterUserDto {
    @NotBlank
    private String fullName;

    @NotBlank
    @Email
    private String email;

    @NotBlank
    @Size(min = 8, max = 72)
    private String password;

    // getters/setters
}

public class LoginUserDto {
    @NotBlank
    @Email
    private String email;

    @NotBlank
    private String password;
```

```
    // getters/setters
}
```

CÁCH CHẠY: mvn -q -DskipTests compile

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH: Compile pass, jakarta.validation imports resolve.

PASS / FAIL: PASS khi compile sạch.

LỖI THƯỜNG GẶP / CÁCH XỬ LÝ: Nếu @Email/@NotBlank không resolve, kiểm spring-boot-starter-validation.

Step 2.4 – Tạo LoginResponse và UserResponse

MỤC TIÊU: Trả contract an toàn: login trả token/TTL; user API không trả password hash.

KIẾN THỨC NỀN CẦN NHỚ: Entity không nên được serialize trực tiếp khi có field nhạy cảm.

CODE / CẤU HÌNH:

```
public record LoginResponse(String token, long expiresIn) {}

public record UserResponse(
    Integer id,
    String fullName,
    String email,
    Date createdAt,
    Date updatedAt
) {}
```

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH: Search project: không có endpoint nào return trực tiếp User entity ở public response.

PASS / FAIL: PASS khi password không nằm trong response DTO.

LỖI THƯỜNG GẶP / CÁCH XỬ LÝ: Đừng "ẩn" password bằng frontend; phải loại khỏi backend response contract.

Step 2.5 – Tạo UserRepository

MỤC TIÊU: Tra cứu User theo email và kiểm duplicate.

KIẾN THỨC NỀN CẦN NHỚ: Spring Data sinh implementation từ repository interface; method name phải khớp entity property.

CODE / CẤU HÌNH:

```
@Repository
public interface UserRepository extends JpaRepository<User, Integer> {
    Optional<User> findByEmail(String email);
    boolean existsByEmail(String email);
}
```

CÁCH CHẠY: Restart app hoặc compile.

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH: App start không báo PropertyReferenceException; repository bean được tạo.

PASS / FAIL: PASS khi startup sạch.

LỖI THƯỜNG GẶP / CÁCH XỬ LÝ: Tên method findByEmail phải khớp field email, không phải DB column name khác.

8. ACTIVITY 3 – CẤU HÌNH AUTHENTICATION CORE

Step 3.1 – Tạo UserDetailsService

MỤC TIÊU: Cho Spring Security biết cách load user theo email.

KIẾN THỨC NỀN CẦN NHỚ: DaoAuthenticationProvider sẽ gọi UserDetailsService khi xác thực UsernamePasswordAuthenticationToken.

CODE / CẤU HÌNH:

```
@Configuration
public class ApplicationConfiguration {

    private final UserRepository userRepository;

    public ApplicationConfiguration(UserRepository userRepository) {
        this.userRepository = userRepository;
    }
```

```
    @Bean
    UserDetailsService userDetailsService() {
        return username -> userRepository.findByEmail(username)
            .orElseThrow(() ->
                new UsernameNotFoundException("User not found"));
    }
}
```

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH: Đặt breakpoint trong lambda loadUserByUsername; sau bước login phải hit.

PASS / FAIL: PASS khi Spring tạo UserDetailsService bean.

LỖI THƯỜNG GẶP / CÁCH XỬ LÝ: Không query bằng fullName nếu login identifier là email.

Step 3.2 – Tạo PasswordEncoder

MỤC TIÊU: Hash password khi signup và verify password khi login.

KIẾN THỨC NỀN CẦN NHỚ: Password hashing là one-way; application so khớp bằng matches(), không giải mã hash.

CODE / CẤU HÌNH:

```
@Bean
PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
```

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH: Bean start thành công; sau signup DB password bắt đầu bằng BCrypt prefix thường gặp và khác plaintext.

PASS / FAIL: PASS khi plaintext không xuất hiện trong cột password.

LỖI THƯỜNG GẶP / CÁCH XỬ LÝ: Không tự dùng SHA-256 đơn thuần cho password; không gọi encode() rồi so sánh chuỗi thủ công ở login.

Step 3.3 – Tạo AuthenticationProvider

MỤC TIÊU: Kết hợp UserDetailsService và PasswordEncoder cho username/password authentication.

KIẾN THỨC NỀN CẦN NHỚ: DaoAuthenticationProvider thực hiện flow load user -> kiểm password -> tạo authenticated token.

CODE / CẤU HÌNH:

```
@Bean
AuthenticationProvider authenticationProvider() {
    DaoAuthenticationProvider provider = new DaoAuthenticationProvider();
    provider.setUserDetailsService(userDetailsService());
    provider.setPasswordEncoder(passwordEncoder());
    return provider;
}
```

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH: Application start không bean error.

PASS / FAIL: PASS khi provider bean tồn tại.

LỖI THƯỜNG GẶP / CÁCH XỬ LÝ: Nếu login luôn fail dù password đúng, kiểm encoder lúc signup và provider dùng cùng PasswordEncoder.

Step 3.4 – Expose AuthenticationManager

MỤC TIÊU: Cho AuthenticationService gọi Spring Security authentication pipeline.

KIẾN THỨC NỀN CẦN NHỚ: AuthenticationManager không tự query DB; nó delegate sang AuthenticationProvider.

CODE / CẤU HÌNH:

```
@Bean
AuthenticationManager authenticationManager(
        AuthenticationConfiguration configuration) throws Exception {
    return configuration.getAuthenticationManager();
}
```

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH: Application context tạo bean không lỗi.

PASS / FAIL: PASS khi AuthenticationService inject được AuthenticationManager.

LỖI THƯỜNG GẶP / CÁCH XỬ LÝ: Không new ProviderManager thủ công nếu chưa hiểu provider chain.

9. ACTIVITY 4 – JWT SERVICE: GENERATE, PARSE, VALIDATE

Step 4.1 – Tạo JwtService

MỤC TIÊU: Đóng gói mọi logic tạo và xác minh JWT.

KIẾN THỨC NỀN CẦN NHỚ: JWT signature phải được verify trước khi tin claims. `sub` dùng làm email/username. `iat` và `exp` phục vụ vòng đời token.

CODE / CẤU HÌNH:

```
@Service
public class JwtService {

    @Value("${security.jwt.secret-key}")
    private String secretKey;

    @Value("${security.jwt.expiration-time}")
    private long jwtExpiration;

    public String extractUsername(String token) {
        return extractClaim(token, Claims::getSubject);
    }

    public <T> T extractClaim(String token, Function<Claims,T> resolver) {
        return resolver.apply(extractAllClaims(token));
    }

    public String generateToken(UserDetails userDetails) {
        return generateToken(new HashMap<>(), userDetails);
    }

    public String generateToken(Map<String,Object> extraClaims, UserDetails userDetails) {
        return Jwts.builder()
            .claims(extraClaims)
            .subject(userDetails.getUsername())
            .issuedAt(new Date(System.currentTimeMillis()))
            .expiration(new Date(System.currentTimeMillis() + jwtExpiration))
            .signWith(getSignInKey())
            .compact();
    }

    public boolean isTokenValid(String token, UserDetails userDetails) {
        return extractUsername(token).equals(userDetails.getUsername())
            && !isTokenExpired(token);
    }

    public long getExpirationTime() { return jwtExpiration; }

    private boolean isTokenExpired(String token) {
        return extractClaim(token, Claims::getExpiration).before(new Date());
    }

    private Claims extractAllClaims(String token) {
        return Jwts.parser()
            .verifyWith(getSignInKey())
            .build()
            .parseSignedClaims(token)
            .getPayload();
    }

    private SecretKey getSignInKey() {
        byte[] keyBytes = Decoders.BASE64.decode(secretKey);
        return Keys.hmacShaKeyFor(keyBytes);
    }
}
```

CÁCH CHẠY: Compile với JJWT 0.12.6.

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH: IDE không báo deprecated/missing methods theo version hiện tại.

PASS / FAIL: PASS khi compile. FAIL nếu method API khác: kiểm version pom.xml trước, không sửa ngẫu nhiên.

LỖI THƯỜNG GẶP / CÁCH XỬ LÝ: JWT_SECRET phải là Base64 phù hợp nếu code dùng Decoders.BASE64; nếu dùng raw text phải thay key-building strategy nhất quán.

Step 4.2 – Hiểu token trước khi dùng

MỤC TIÊU: Củng cố cấu trúc Header.Payload.Signature và tránh hiểu sai "JWT là mã hóa".

KIẾN THỨC NỀN CẦN NHỚ: Payload có thể decode bằng client; không đặt password, DB secret hay dữ liệu nhạy cảm vào claims.

THAO TÁC:

- Sau khi login ở bước sau, copy token.
- Decode header/payload bằng công cụ local/JWT debugger chỉ với dữ liệu lab.
- Quan sát sub, iat, exp.

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH: sub = email user; exp > iat; token có 3 phần phân cách bằng dấu chấm.

PASS / FAIL: PASS nếu claims đúng và signature được server verify.

LỖI THƯỜNG GẶP / CÁCH XỬ LÝ: Decode thành công không chứng minh token hợp lệ; signature verification mới chứng minh integrity/authenticity.

Step 4.3 – Tamper test ở mức JwtService

MỤC TIÊU: Chứng minh chữ ký phát hiện token bị sửa.

KIẾN THỨC NỀN CẦN NHỚ: Chỉ sửa một ký tự payload/signature cũng phải làm verification fail.

THAO TÁC:

- Generate token hợp lệ.
- Sửa một ký tự nhưng giữ format.
- Gọi extract/validate trong test hoặc request protected.

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH: JJWT throw JWT/signature exception hoặc request được map thành 401.

PASS / FAIL: PASS khi tampered token không bao giờ tạo authenticated SecurityContext.

LỖI THƯỜNG GẶP / CÁCH XỬ LÝ: Không catch exception rồi "return true" hoặc tiếp tục filter như authenticated.

10. ACTIVITY 5 – JWT FILTER VÀ SECURITY FILTER CHAIN

Step 5.1 – Tạo JwtAuthenticationFilter

MỤC TIÊU: Đọc Authorization header, xác minh JWT và set Authentication vào SecurityContext.

KIẾN THỨC NỀN CẦN NHỚ: Filter chỉ xác thực. Quyền truy cập URL nằm ở SecurityFilterChain/method security.

CODE / CẤU HÌNH:

```
@Component
public class JwtAuthenticationFilter extends OncePerRequestFilter {

    private final JwtService jwtService;
    private final UserDetailsService userDetailsService;

    public JwtAuthenticationFilter(JwtService jwtService,
                                   UserDetailsService userDetailsService) {
        this.jwtService = jwtService;
        this.userDetailsService = userDetailsService;
    }

    @Override
    protected void doFilterInternal(
            HttpServletRequest request,
            HttpServletResponse response,
            FilterChain filterChain) throws ServletException, IOException {

        String authHeader = request.getHeader("Authorization");

        if (authHeader == null || !authHeader.startsWith("Bearer ")) {
            filterChain.doFilter(request, response);
            return;
```

```
        }

        String jwt = authHeader.substring(7);

        try {
            String email = jwtService.extractUsername(jwt);

            if (email != null &&
                SecurityContextHolder.getContext().getAuthentication() == null) {

                UserDetails userDetails =
                    userDetailsService.loadUserByUsername(email);

                if (jwtService.isTokenValid(jwt, userDetails)) {
                    UsernamePasswordAuthenticationToken authToken =
                        new UsernamePasswordAuthenticationToken(
                            userDetails, null, userDetails.getAuthorities());

                    authToken.setDetails(
                        new WebAuthenticationDetailsSource().buildDetails(request));

                    SecurityContextHolder.getContext()
                        .setAuthentication(authToken);
                }
            }

            filterChain.doFilter(request, response);
        } catch (JwtException | IllegalArgumentException ex) {
            response.sendError(HttpServletResponse.SC_UNAUTHORIZED,
                               "Invalid or expired token");
        }
    }
}
```

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH: Đặt breakpoint tại authHeader, email, setAuthentication. Protected request với token hợp lệ phải đi qua đủ checkpoint.

PASS / FAIL: PASS khi SecurityContext có authenticated principal trước Controller.

LỖI THƯỜNG GẶP / CÁCH XỬ LÝ: Quên filterChain.doFilter() làm request bị "kẹt"; set Authentication trước khi verify token là lỗi nghiêm trọng.

Step 5.2 – Tạo SecurityConfiguration / SecurityFilterChain

MỤC TIÊU: Định nghĩa endpoint public/protected và stateless session policy.

KIẾN THỨC NỀN CẦN NHỚ: JWT bearer API thường không dùng server HTTP session cho authentication state; Core Lab dùng STATELESS.

CODE / CẤU HÌNH:

```
@Configuration
@EnableWebSecurity
public class SecurityConfiguration {

    private final JwtAuthenticationFilter jwtAuthenticationFilter;
    private final AuthenticationProvider authenticationProvider;

    public SecurityConfiguration(
            JwtAuthenticationFilter jwtAuthenticationFilter,
            AuthenticationProvider authenticationProvider) {
        this.jwtAuthenticationFilter = jwtAuthenticationFilter;
        this.authenticationProvider = authenticationProvider;
    }

    @Bean
    SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
            .csrf(csrf -> csrf.disable())
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/auth/**").permitAll()
                .anyRequest().authenticated()
            )
            .sessionManagement(session -> session
                .sessionCreationPolicy(SessionCreationPolicy.STATELESS)
```

```
            )
            .authenticationProvider(authenticationProvider)
            .addFilterBefore(
                jwtAuthenticationFilter,
                UsernamePasswordAuthenticationFilter.class
            );

        return http.build();
    }
}
```

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH: App start. POST /auth/login phải không cần token; GET /users/me không token phải bị từ chối.

PASS / FAIL: PASS khi public/protected boundary đúng.

LỖI THƯỜNG GẶP / CÁCH XỬ LÝ: `csrf.disable()` chỉ là lab/stateless bearer context, không phải quy tắc "API luôn disable CSRF" trong mọi architecture.

Step 5.3 – Tạo checkpoint cho 401 trước khi viết Controller

MỤC TIÊU: Chứng minh security boundary hoạt động độc lập với business controller.

KIẾN THỨC NỀN CẦN NHỚ: Nếu URL protected không có matching controller có thể trả 404 sau security; nên test trên endpoint có thật sau bước 7. Tuy nhiên có thể kiểm filter chain bằng MockMvc sau này.

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH: Ghi lại kỳ vọng: /auth/** permitAll; /users/** authenticated.

PASS / FAIL: PASS khi policy được code review rõ ràng trước khi thêm endpoint.

LỖI THƯỜNG GẶP / CÁCH XỬ LÝ: Không dùng `.anyRequest().permitAll()` chỉ để "cho chạy".

11. ACTIVITY 6 – AUTHENTICATION SERVICE VÀ CONTROLLER

Step 6.1 – Tạo AuthenticationService.signup()

MỤC TIÊU: Đăng ký user mới, kiểm duplicate và hash password.

KIẾN THỨC NỀN CẦN NHỚ: Pre-check duplicate cải thiện error message; UNIQUE constraint DB vẫn là lớp bảo vệ cuối cùng.

CODE / CẤU HÌNH:

```
@Service
public class AuthenticationService {
    private final UserRepository userRepository;
    private final PasswordEncoder passwordEncoder;
    private final AuthenticationManager authenticationManager;

    public AuthenticationService(UserRepository userRepository,
                                 PasswordEncoder passwordEncoder,
                                 AuthenticationManager authenticationManager) {
        this.userRepository = userRepository;
        this.passwordEncoder = passwordEncoder;
        this.authenticationManager = authenticationManager;
    }

    public User signup(RegisterUserDto input) {
        if (userRepository.existsByEmail(input.getEmail())) {
            throw new IllegalStateException("Email already exists");
        }

        User user = new User();
        user.setFullName(input.getFullName().trim());
        user.setEmail(input.getEmail().trim().toLowerCase());
        user.setPassword(passwordEncoder.encode(input.getPassword()));

        return userRepository.save(user);
    }

    public User authenticate(LoginUserDto input) {
        authenticationManager.authenticate(
            new UsernamePasswordAuthenticationToken(
                input.getEmail(), input.getPassword()));
```

```
            return userRepository.findByEmail(input.getEmail())
                .orElseThrow();
        }
    }
```

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH: Breakpoint: trước save password đã là hash; duplicate email throw trước save.

PASS / FAIL: PASS khi DB không bao giờ nhận plaintext password.

LỖI THƯỜNG GẶP / CÁCH XỬ LÝ: Không encode password trong authenticate(); DaoAuthenticationProvider tự dùng encoder.matches().

Step 6.2 – Tạo AuthenticationController

MỤC TIÊU: Expose signup/login qua REST.

KIẾN THỨC NỀN CẦN NHỚ: Controller chỉ phối hợp HTTP + Service/JWT, không query repository trực tiếp.

CODE / CẤU HÌNH:

```
@RestController
@RequestMapping("/auth")
public class AuthenticationController {

    private final JwtService jwtService;
    private final AuthenticationService authenticationService;

    public AuthenticationController(JwtService jwtService,
                                    AuthenticationService authenticationService) {
        this.jwtService = jwtService;
        this.authenticationService = authenticationService;
    }

    @PostMapping("/signup")
    public ResponseEntity<UserResponse> register(
            @Valid @RequestBody RegisterUserDto dto) {
        User user = authenticationService.signup(dto);
        return ResponseEntity.status(HttpStatus.CREATED)
            .body(toResponse(user));
    }

    @PostMapping("/login")
    public ResponseEntity<LoginResponse> authenticate(
            @Valid @RequestBody LoginUserDto dto) {
        User user = authenticationService.authenticate(dto);
        String token = jwtService.generateToken(user);
        return ResponseEntity.ok(
            new LoginResponse(token, jwtService.getExpirationTime()));
    }

    private UserResponse toResponse(User user) {
        return new UserResponse(
            user.getId(), user.getFullName(), user.getEmail(),
            user.getCreatedAt(), user.getUpdatedAt());
    }
}
```

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH: Compile và startup; Spring mapping log có /auth/signup và /auth/login.

PASS / FAIL: PASS khi Controller không expose password.

LỖI THƯỜNG GẶP / CÁCH XỬ LÝ: Nếu login endpoint trả 403/401 dù permitAll, kiểm request path và SecurityFilterChain matcher.

Step 6.3 – Global exception/validation handling cơ bản

MỤC TIÊU: Trả lỗi client dễ hiểu thay vì stack trace HTML/generic.

KIẾN THỨC NỀN CẦN NHỚ: Authentication failures, duplicate email và Bean Validation là nhóm lỗi khác nhau.

CODE / CẤU HÌNH:

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(IllegalStateException.class)
```

```
    ResponseEntity<Map<String,Object>> conflict(IllegalStateException ex) {
        return ResponseEntity.status(HttpStatus.CONFLICT)
            .body(Map.of("message", ex.getMessage()));
    }

    @ExceptionHandler(BadCredentialsException.class)
    ResponseEntity<Map<String,Object>> badCredentials(BadCredentialsException ex) {
        return ResponseEntity.status(HttpStatus.UNAUTHORIZED)
            .body(Map.of("message", "Invalid email or password"));
    }
}
```

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH: Duplicate signup -> 409; bad login -> 401; response không chứa stack trace/password.

PASS / FAIL: PASS khi status code phản ánh đúng failure category.

LỖI THƯỜNG GẶP / CÁCH XỬ LÝ: Không trả "email không tồn tại" khác "password sai" nếu muốn giảm account enumeration; Core Lab có thể dùng message chung.

Step 6.4 – Chạy smoke test public endpoints

MỤC TIÊU: Chứng minh signup/login hoạt động trước protected endpoint.

KIẾN THỨC NỀN CẦN NHỚ: Debug theo tầng: HTTP -> Controller -> Service -> AuthManager -> DB.

CÁCH CHẠY: mvn spring-boot:run; dùng Postman bước 8.2–8.3 bên dưới.

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH: Signup 201, Login 200 + token.

PASS / FAIL: PASS trước khi sang protected API.

LỖI THƯỜNG GẶP / CÁCH XỬ LÝ: Nếu signup được nhưng login fail, so DB hash, email normalization và PasswordEncoder provider.

12. ACTIVITY 7 – PROTECTED USER ENDPOINTS

Step 7.1 – Tạo UserService

MỤC TIÊU: Tách logic đọc user khỏi Controller.

KIẾN THỨC NỀN CẦN NHỚ: Service không nên trả password field cho response layer.

CODE / CẤU HÌNH:

```
@Service
public class UserService {
    private final UserRepository userRepository;

    public UserService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    public List<User> allUsers() {
        return userRepository.findAll();
    }
}
```

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH: Compile; bean inject được.

PASS / FAIL: PASS khi Controller không gọi Repository trực tiếp.

LỖI THƯỜNG GẶP / CÁCH XỬ LÝ: Ở production nên trả projection/DTO sớm thay vì User entity list.

Step 7.2 – Tạo /users/me

MỤC TIÊU: Trả thông tin principal hiện tại dựa trên SecurityContext.

KIẾN THỨC NỀN CẦN NHỚ: Sau JwtAuthenticationFilter, @AuthenticationPrincipal có thể nhận UserDetails principal.

CODE / CẤU HÌNH:

```
@RestController
@RequestMapping("/users")
public class UserController {
```

```
    private final UserService userService;

    public UserController(UserService userService) {
        this.userService = userService;
    }

    @GetMapping("/me")
    public ResponseEntity<UserResponse> me(
            @AuthenticationPrincipal User user) {
        return ResponseEntity.ok(toResponse(user));
    }

    @GetMapping
    public List<UserResponse> allUsers() {
        return userService.allUsers().stream()
            .map(this::toResponse)
            .toList();
    }

    private UserResponse toResponse(User u) {
        return new UserResponse(
            u.getId(), u.getFullName(), u.getEmail(),
            u.getCreatedAt(), u.getUpdatedAt());
    }
}
```

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH: Không token -> rejected; token hợp lệ -> /users/me trả đúng email.

PASS / FAIL: PASS khi principal đúng user login.

LỖI THƯỜNG GẶP / CÁCH XỬ LÝ: Nếu @AuthenticationPrincipal null, kiểm filter có set SecurityContext và principal type có đúng User.

Step 7.3 – Kiểm response không lộ password

MỤC TIÊU: Bảo đảm API contract an toàn.

KIẾN THỨC NỀN CẦN NHỚ: Hash password vẫn là credential-sensitive; không trả cho client dù không phải plaintext.

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH: Search JSON response của /users/me và /users: không có field password.

PASS / FAIL: PASS nếu không có password/hash.

LỖI THƯỜNG GẶP / CÁCH XỬ LÝ: Không dùng @JsonIgnore như giải pháp duy nhất nếu endpoint vẫn serialize entity; DTO rõ ràng dễ kiểm soát hơn.

13. ACTIVITY 8 – CHẠY VÀ KIỂM THỬ BẰNG POSTMAN

Step 8.1 – Start application và kiểm startup

MỤC TIÊU: Khóa trạng thái server trước test HTTP.

KIẾN THỨC NỀN CẦN NHỚ: Nếu application context fail thì Postman error không có giá trị chẩn đoán security.

CODE / CẤU HÌNH:

```
mvn clean test
mvn spring-boot:run
```

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH: Log có Started Lab06JwtApplication; port 8005 LISTENING.

PASS / FAIL: PASS khi server up.

LỖI THƯỜNG GẶP / CÁCH XỬ LÝ: Đọc exception đầu tiên/root cause, không chỉ dòng cuối stack trace.

Step 8.2 – POST /auth/signup

MỤC TIÊU: Đăng ký user test.

KIẾN THỨC NỀN CẦN NHỚ: Đây là public endpoint nhưng phải validation + encode password.

CODE / CẤU HÌNH:

```
POST {{baseUrl}}/auth/signup
Content-Type: application/json

{
```

```
  "fullName": "SBA301 Student",
  "email": "student01@fpt.edu.vn",
  "password": "P@ssword123"
}
```

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH: HTTP 201; body có id/fullName/email; SQL Server có row mới; DB password khác P@ssword123.

PASS / FAIL: PASS khi cả HTTP + DB cùng đúng.

LỖI THƯỜNG GẶP / CÁCH XỬ LÝ: Nếu 403: /auth/** chưa permitAll hoặc CSRF/config mismatch; nếu 500: xem SQL/constraint.

Step 8.3 – POST /auth/login và lưu token

MỤC TIÊU: Xác thực credential và nhận JWT.

KIẾN THỨC NỀN CẦN NHỚ: AuthenticationManager xử lý password; JwtService chỉ phát token sau khi authentication thành công.

CODE / CẤU HÌNH:

```
POST {{baseUrl}}/auth/login
Content-Type: application/json

{
  "email": "student01@fpt.edu.vn",
  "password": "P@ssword123"
}
```

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH: HTTP 200; body có token và expiresIn.

PASS / FAIL: PASS khi token có 3 phần và expiresIn > 0.

LỖI THƯỜNG GẶP / CÁCH XỬ LÝ: Sai password phải 401, không phải 200 với token.

Trong Postman Tests tab của request login:

```
const json = pm.response.json();
pm.environment.set("token", json.token);
```

Step 8.4 – GET /users/me với Bearer token

MỤC TIÊU: Chứng minh JWT filter + SecurityContext + Controller hoạt động end-to-end.

KIẾN THỨC NỀN CẦN NHỚ: Header chuẩn là `Authorization: Bearer <token>`; chữ Bearer và dấu cách quan trọng.

CODE / CẤU HÌNH:

```
GET {{baseUrl}}/users/me
Authorization: Bearer {{token}}
```

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH: HTTP 200; email trong response = student01@fpt.edu.vn.

PASS / FAIL: PASS khi đúng principal.

LỖI THƯỜNG GẶP / CÁCH XỬ LÝ: 401: token missing/expired/signature invalid; 403 trong Core Lab thường do authorization rule khác dự kiến.

Step 8.5 – GET /users

MỤC TIÊU: Kiểm endpoint authenticated list.

KIẾN THỨC NỀN CẦN NHỚ: Core Lab chưa role restriction; bất kỳ authenticated user hợp lệ có thể truy cập theo policy hiện tại.

CODE / CẤU HÌNH:

```
GET {{baseUrl}}/users
Authorization: Bearer {{token}}
```

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH: 200 + array/list; không có password.

PASS / FAIL: PASS nếu response safe.

LỖI THƯỜNG GẶP / CÁCH XỬ LÝ: Phần Advanced sẽ đổi /users thành ADMIN-only để phân biệt 401/403.

Step 8.6 – Negative: không token / malformed / tampered token

MỤC TIÊU: Chứng minh fail-closed behavior.

KIẾN THỨC NỀN CẦN NHỚ: Security đúng phải từ chối credential thiếu/sai, không "bỏ qua filter rồi permit".

THAO TÁC:

- Gọi /users/me không Authorization.
- Gửi `Bearer abc`.
- Sửa 1 ký tự token hợp lệ rồi gửi lại.

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH: Tất cả request protected phải không trả 200.

PASS / FAIL: PASS khi status là 401 hoặc security contract đã định nghĩa, và Controller business không chạy.

LỖI THƯỜNG GẶP / CÁCH XỬ LÝ: Nếu tampered token vẫn 200: lỗi nghiêm trọng ở verification/filter context.

Step 8.7 – Negative: expired token

MỤC TIÊU: Chứng minh exp được enforce.

KIẾN THỨC NỀN CẦN NHỚ: Có thể tạm giảm expiration-time xuống 5000 ms trong profile lab, login, chờ >5s rồi gọi protected endpoint.

THAO TÁC:

- Đổi TTL tạm thời.
- Restart app.
- Login lấy token.
- Chờ hết hạn.
- GET /users/me.

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH: Expired token bị từ chối.

PASS / FAIL: PASS khi không 200.

LỖI THƯỜNG GẶP / CÁCH XỬ LÝ: Khôi phục TTL 3600000 sau experiment.

Step 8.8 – Negative: duplicate signup và bad credential

MỤC TIÊU: Kiểm business/auth failure contract.

KIẾN THỨC NỀN CẦN NHỚ: Duplicate resource khác authentication failure nên status khác nhau.

THAO TÁC:

- Signup lại cùng email.
- Login đúng email nhưng password sai.

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH: Duplicate -> 409; bad credential -> 401.

PASS / FAIL: PASS khi DB không tạo duplicate và login sai không phát token.

LỖI THƯỜNG GẶP / CÁCH XỬ LÝ: Nếu duplicate thành 500, cần map DataIntegrityViolation/BusinessConflict phù hợp.

14. TRACE END-TO-END – SINH VIÊN PHẢI GIẢI THÍCH ĐƯỢC

14.1. Login trace

```
POST /auth/login
 -> AuthenticationController.authenticate(dto)
 -> AuthenticationService.authenticate(dto)
 -> AuthenticationManager.authenticate(...)
 -> ProviderManager
 -> DaoAuthenticationProvider
 -> UserDetailsService.loadUserByUsername(email)
 -> UserRepository.findByEmail(email)
 -> PasswordEncoder.matches(raw, encoded)
 <- authenticated Authentication
 -> JwtService.generateToken(user)
 <- 200 { token, expiresIn }
```

14.2. Protected request trace

```
GET /users/me + Bearer JWT
 -> SecurityFilterChain
 -> JwtAuthenticationFilter
 -> parse + verify signature + exp
 -> extract subject/email
 -> UserDetailsService.loadUserByUsername(email)
 -> JwtService.isTokenValid(...)
 -> SecurityContextHolder.setAuthentication(...)
 -> authorization rule authenticated() passes
 -> UserController.me(principal)
 <- 200 UserResponse
```

EVIDENCE: Sinh viên đặt breakpoint ở JwtAuthenticationFilter và AuthenticationService để chỉ ra request đang ở giai đoạn nào.

15. TEST MATRIX TỔNG HỢP

ID	Request/Case	Expected	DB/Other Evidence
T01	POST signup valid	201	new user; password hashed
T02	POST signup duplicate	409	no duplicate row
T03	POST login valid	200 + JWT	no DB mutation required
T04	POST login bad password	401	no token
T05	GET /users/me no token	401	Controller not reached
T06	GET /users/me valid token	200	correct principal
T07	GET /users valid token	200	no password field
T08	Malformed token	401	no SecurityContext auth
T09	Tampered token	401	signature reject
T10	Expired token	401	exp reject
T11	Validation invalid email	400	no user row
T12	Empty password signup	400	no user row

16. DEBUGGING – LỖI THƯỜNG GẶP VÀ CÁCH XỬ LÝ

Triệu chứng	Nguyên nhân thường gặp	Cách xác minh	Fix
App không start	DB/config/bean cycle	root cause + datasource URL	sửa config trước JWT
Signup 403	/auth chưa permitAll / CSRF	SecurityFilterChain	permit /auth/**, kiểm CSRF context
Login luôn 401	encoder/provider/email mismatch	breakpoint provider + DB hash	cùng PasswordEncoder, normalize email
Protected luôn 401	header/token/filter	Network header + filter breakpoint	Bearer prefix, key, exp
Tampered token gây 500	exception chưa map	stack trace	catch JwtException/map 401
Principal null	SecurityContext chưa set	filter breakpoint	setAuthentication sau validate
User response có password	serialize entity	inspect JSON	dùng UserResponse DTO
SignatureException	secret decode/key mismatch	same env variable?	fix Base64/key strategy
ExpiredJwtException	TTL hết	inspect exp	login lại hoặc TTL test
401 vs 403 lẫn lộn	authn/authz handler	security logs	chuẩn hóa contract advanced

16.1. Debugging decision tree

```
Protected endpoint failed
|
+-- Server started?
|    +-- no -> datasource/beans/config
|
+-- Authorization header present?
|    +-- no -> expected 401
|
+-- Header starts with "Bearer "?
|    +-- no -> 401
|
+-- JWT parse/signature/exp valid?
|    +-- no -> 401
|
+-- User found by subject/email?
|    +-- no -> 401
```

```
|
+-- SecurityContext Authentication set?
|    +-- no -> filter bug
|
+-- Authorization rule passes?
     +-- no -> 403
     +-- yes -> Controller/business layer
```

17. BREAK-IT → DIAGNOSE → FIX WORKSHOP

Break intentionally	Expected symptom	Diagnose evidence	Fix
Sai DB name	startup fail	SQLException root cause	correct datasource URL
JWT secret quá ngắn/sai Base64	key/parser error	startup/request stack trace	generate correct key format
Bỏ passwordEncoder ở signup	login fail	DB plaintext + provider mismatch	encode before save
Bỏ permitAll(/auth/**)	signup/login denied	security trace	restore matcher
Bỏ addFilterBefore	JWT không authenticate	filter breakpoint never hit	register filter
Set SecurityContext trước validate	tampered token risk	code review	validate first
Return User entity	password hash leaks	response JSON	map DTO
TTL 1 ms	immediate expired	exp claim	restore TTL

18. AUTOMATED SECURITY TESTS – COURSE-READY

Step 18.1 – MockMvc: protected endpoint without token

MỤC TIÊU: Đưa security contract vào automated test.

KIẾN THỨC NỀN CẦN NHỚ: Postman là manual evidence; regression cần test có thể chạy lại tự động.

CODE / CẤU HÌNH:

```
@SpringBootTest
@AutoConfigureMockMvc
class SecurityContractTest {

    @Autowired MockMvc mvc;

    @Test
    void usersMe_withoutToken_isUnauthorized() throws Exception {
        mvc.perform(get("/users/me"))
           .andExpect(status().isUnauthorized());
    }
}
```

CÁCH CHẠY: mvn test

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH: Test xanh.

PASS / FAIL: PASS khi missing token regression được bắt.

LỖI THƯỜNG GẶP / CÁCH XỬ LÝ: Nếu app contract hiện trả 403 do default handler, cần chuẩn hóa 401 contract thay vì sửa test mù quáng.

Step 18.2 – Integration test login → token → protected endpoint

MỤC TIÊU: Test full security pipeline.

KIẾN THỨC NỀN CẦN NHỚ: Test nên tạo user bằng encoded password, login qua HTTP, parse token rồi gọi protected endpoint.

THAO TÁC:

- Arrange user test.
- POST /auth/login bằng MockMvc.
- Extract JSON token.
- GET /users/me + Authorization Bearer.
- Assert 200 + email.
- Tamper token và assert unauthorized.

CÁCH KIỂM TRA ĐÃ HOÀN THÀNH: Happy path + negative path cùng pass.

PASS / FAIL: PASS khi không phụ thuộc thứ tự test/DB rác.

LỖI THƯỜNG GẶP / CÁCH XỬ LÝ: Dùng unique email per test hoặc cleanup transaction/test DB.

19. CORE LAB INTEGRATION GATE G1–G16

Gate	Yêu cầu	Evidence
G1	JDK/Maven/SQL Server pass	tool output
G2	Project build sạch	mvn compile
G3	Datasource connect	startup log
G4	users table/mapping đúng	SQL query
G5	Password encoded	DB hash != plaintext
G6	UserDetailsService works	debug trace
G7	AuthenticationProvider works	valid/bad login
G8	JwtService generate/verify	token/tamper evidence
G9	JwtAuthenticationFilter works	breakpoint/SecurityContext
G10	/auth/** public	signup/login no token
G11	/users/** protected	no token rejected
G12	/users/me principal đúng	200 + email
G13	No password leakage	response inspection
G14	Negative matrix pass	Postman
G15	Automated test pass	mvn test
G16	Student explains flow	oral/trace diagram

20. CHECKLIST HOÀN THÀNH LAB 06

- ☐ Project Spring Boot + dependencies build được.
- ☐ SQL Server datasource hoạt động.
- ☐ JWT secret không hard-code vào Git.
- ☐ User implements UserDetails theo course baseline.
- ☐ Password BCrypt encode trước khi save.
- ☐ UserRepository findByEmail/existsByEmail hoạt động.
- ☐ AuthenticationManager/Provider hoạt động.
- ☐ JwtService generate/parse/validate đúng.
- ☐ JwtAuthenticationFilter đọc Bearer token đúng.
- ☐ SecurityFilterChain stateless + public/protected đúng.
- ☐ /auth/signup chạy và duplicate/validation có status hợp lý.
- ☐ /auth/login trả JWT với credential đúng và 401 khi sai.
- ☐ /users/me với JWT chạy.
- ☐ /users/me không JWT bị từ chối.
- ☐ Tampered/expired token bị từ chối.
- ☐ /users response không chứa password/hash.
- ☐ Postman environment có baseUrl/token.
- ☐ Có negative-test evidence.
- ☐ Có ít nhất một automated security test.
- ☐ Sinh viên giải thích được login trace và protected-request trace.

21. BỘ BẰNG CHỨNG KHUYẾN NGHỊ SINH VIÊN NỘP

Evidence	Nội dung
E1	Ảnh/console `mvn test` pass
E2	Startup log backend ở port 8005
E3	SQL users row + password hash (che dữ liệu nhạy cảm nếu cần)
E4	Postman signup 201
E5	Postman login 200 + token (che phần lớn token khi đưa vào report)

Evidence	Nội dung
E6	/users/me 200 với Bearer token
E7	No-token/tampered/expired negative evidence
E8	Debugger screenshot/note trong JwtAuthenticationFilter
E9	Security flow diagram hoặc sequence note
E10	Break-It → Diagnose → Fix ít nhất 2 lỗi

22. QUICK RUN CARD – CHẠY LẠI PROJECT SAU KHI ĐÃ HOÀN THÀNH

```
1) Start SQL Server

2) Set environment variables
   DB_USER=...
   DB_PASSWORD=...
   JWT_SECRET=...

3) Test
   mvn clean test

4) Run
   mvn spring-boot:run

5) Verify public endpoints
   POST http://localhost:8005/auth/signup
   POST http://localhost:8005/auth/login

6) Save JWT to Postman token variable

7) Verify protected endpoints
   GET http://localhost:8005/users/me
   GET http://localhost:8005/users
   Header: Authorization: Bearer {{token}}

8) Verify negative
   no token / bad password / tampered / expired
```

23. ADVANCED EXTENSION & REFACTOR TRACK – TỰ NGHIÊN CỨU

ADVANCED: Không bắt buộc để pass Core Lab. Mỗi track nên nộp: mục tiêu → thiết kế → code diff → test evidence → security/trade-off note.

23.1. Role/Authority & Authorization

Thêm USER/ADMIN; `/users/me` authenticated, `/users` ADMIN-only. Kiểm USER→403, ADMIN→200. Không cho signup tự chọn ADMIN.

23.2. Chuẩn hóa 401 vs 403

Dùng AuthenticationEntryPoint và AccessDeniedHandler để trả JSON thống nhất. Giải thích 401 = chưa/không xác thực, 403 = đã xác thực nhưng thiếu quyền.

23.3. Refresh Token + Rotation

Access token TTL ngắn; refresh token persisted/rotated. Test reuse token cũ bị từ chối. Không coi một JWT thứ hai là refresh token đầy đủ nếu không có revocation/replay strategy.

23.4. Logout / Revocation

So sánh short-lived access token + refresh revocation, denylist `jti`, token version/user session.

23.5. Claims governance

Thêm `iss`, `aud`, `jti`, roles/scopes có chủ đích; validate issuer/audience; không đưa dữ liệu nhạy cảm vào payload.

23.6. HMAC → RSA/ECDSA + JWK/JWKS

Sign bằng private key, verify bằng public key; nghiên cứu `kid` và key rotation.

23.7. Refactor custom JWT filter → OAuth2 Resource Server
So sánh custom JJWT với Spring Security Resource Server: standards, error handling, issuer/JWK, code size.

23.8. Password security
So sánh BCrypt/Argon2/PBKDF2/scrypt; dùng DelegatingPasswordEncoder và đo work factor trên máy lab.

23.9. Tách JPA Entity khỏi UserDetails
Tạo SecurityUser/Principal adapter để domain entity không phụ thuộc Spring Security interfaces.

23.10. Method Security
Thêm @EnableMethodSecurity và @PreAuthorize cho business method; test 403.

23.11. React/Axios integration
Login form, lưu access token theo policy course, Axios interceptor, protected route, CORS và 401 UX.

23.12. Security testing nâng cao
MockMvc roles, invalid signature, expired, issuer/audience, method security, authorization regression suite.

23.13. Audit/Observability
Audit login success/failure nhưng không log password/token đầy đủ; correlation ID/security event.

23.14. Flyway/Liquibase
Thay ddl-auto=update bằng migration + validate; tạo migration users/roles/refresh_tokens.

23.15. Profiles & Secrets
application-dev/test/prod; env/secret store; không commit secret. Viết deployment checklist.

24. NGUỒN VÀ PHẠM VI ÁP DỤNG

Tài liệu này được xây dựng theo cấu trúc workbook của Lab 05 và bám nội dung Lab 06 – Secure REST Services with Spring Boot using JSON Web Token (JWT): Spring Initializr, Spring Web, Spring Data JPA, Spring Security, SQL Server, User/UserDetails, DTO, Repository, JWT classes, Security configuration/filter, Authentication service/controller và kiểm thử signup/login/protected endpoints. Các phần Advanced là mở rộng tự nghiên cứu, không thay đổi yêu cầu Core Lab.